MoE GEMM 算子优化分析

2026 年 4 月 21 日

1 MOE 大致流程回顾

每个 token 根据 top_k_ids 选取自己关心的 k 个专家权重，分别和这些权重操作后根据 top_k 中存储的权重和将结果加权求和。

2 v0：分组调度方法

2.1 优化思路

分组调度（Grouped Launch）的核心思想是将连续的 token 分组，每个线程块处理一组 token，以提高 L2 缓存的数据重用率。

2.2 代码分析

v0 版本的关键代码片段是将不同 token 考虑的专家合并为不同专家考虑的 token，比如：

$\{\{1,2\}\{1,3\}\{2,3\}\}->\{1,2,1,3,2,3\}->\{0,2,1,4,3,5\}$

但下面代码中显示在实现时引入了 $group_size_m$ 参数来进一步分组，可能是结合了后面的列优先?

```
@triton.jit
def fused_moe_kernel(
    ptr_a, ptr_b, ptr_c,
    stride_am, stride_ak,
```

```
    stride_bm, stride_bn,
    stride_cm, stride_cn,
    M, N, K,
    BLOCK_SIZE_M: tl.constexpr,
    BLOCK_SIZE_N: tl.constexpr,
    BLOCK_SIZE_K: tl.constexpr,
    GROUP_SIZE_M: tl.constexpr,
    ...
):
    pid = tl.program_id(axis=0)
    num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
    num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
    num_pid_in_group = GROUP_SIZE_M * num_pid_n

    group_id = pid // num_pid_in_group
    first_pid_m = group_id * GROUP_SIZE_M
    group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)

    pid_m = first_pid_m + ((pid % num_pid_in_group) %
        group_size_m)
    pid_n = (pid % num_pid_in_group) // group_size_m
```

Listing 1: v0 分组调度核心逻辑

2.3 优化原理

1. **分组机制**：通过 $GROUP_SIZE_M$ 参数控制每个组包含的线程块数量
2. **数据局部性**：连续的 token 被分配到同一组，提高 L2 缓存命中率
3. **负载均衡**：通过动态调整组大小，确保每个线程块的工作量均衡

2.4 性能特点

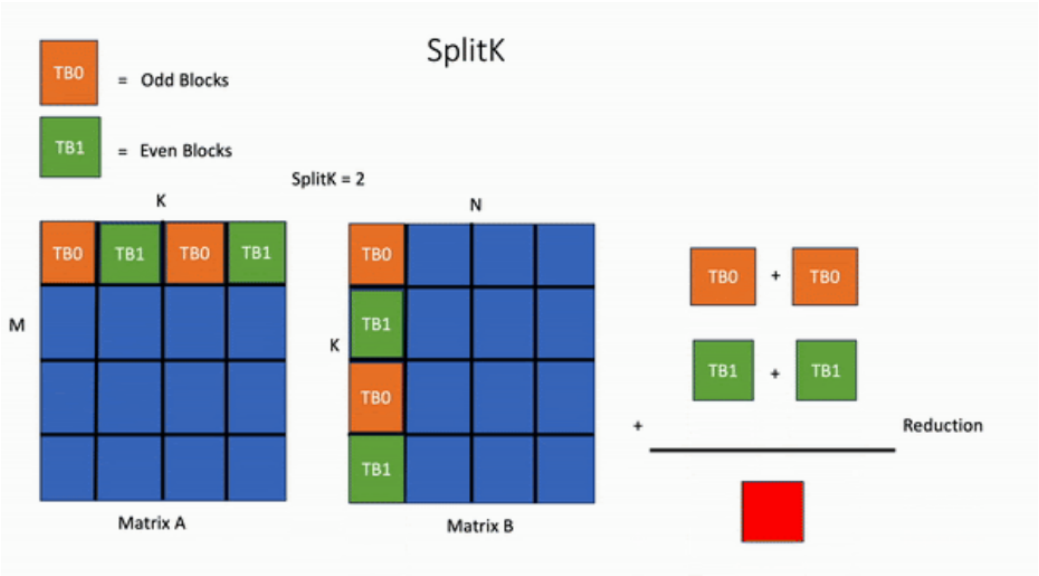
- 适用于小到中等规模的 batch size
- 在 token 分布均匀时表现良好

- 实现相对简单，易于理解和维护

3 v1：Split-K 优化方法

3.1 优化思路

Split-K 是一种工作分解策略，将矩阵乘法的 K 维度分割为多个部分，由多个线程块并行处理。这种方法在 PyTorch 博客中显示可以带来 18-20% 的性能提升。(在实际实现中和前面的分组策略结合使用)



3.2 代码分析

v1 版本的关键代码片段：

```
pid = tl.program_id(axis=0)
    pid_k = tl.program_id(axis=1)

    num_tokens_post_padded = tl.load(num_tokens_post_padded_ptr)

    # print("num_tokens_post_padded: ", num_tokens_post_padded)

```

```
    pid_m, pid_n = grouped_launch(pid,
                                  EM, N,
                                  block_m, block_n, group_m)

    total_blocks_k = tl.cdiv(K, block_k*split_k)
```

Listing 2: v1 Split-K 核心逻辑

3.3 优化原理

1. 并行度从 MN 增加到 MN × SplitK，提高 GPU 利用率
2. 但需要线程块之间额外沟通，引入新开销

4 v2：列主序调度方法

4.1 优化思路

列主序调度是 PyTorch 博客中介绍的最优策略，在 A100 上加速 4x，在 H100 上加速 4.4x。其核心思想是通过改变线程块到输出矩阵的映射方式，提高数据局部性和缓存命中率。

4.2 代码分析

v2 版本的关键代码片段：

```

def col_major(pid,
              m, n,
              block_m: tl.constexpr, block_n: tl.constexpr):

    grid_m = tl.cdiv(m, block_m)
    grid_n = tl.cdiv(n, block_n)

    pid_m = (pid % grid_n)
    pid_n = pid // grid_m
```

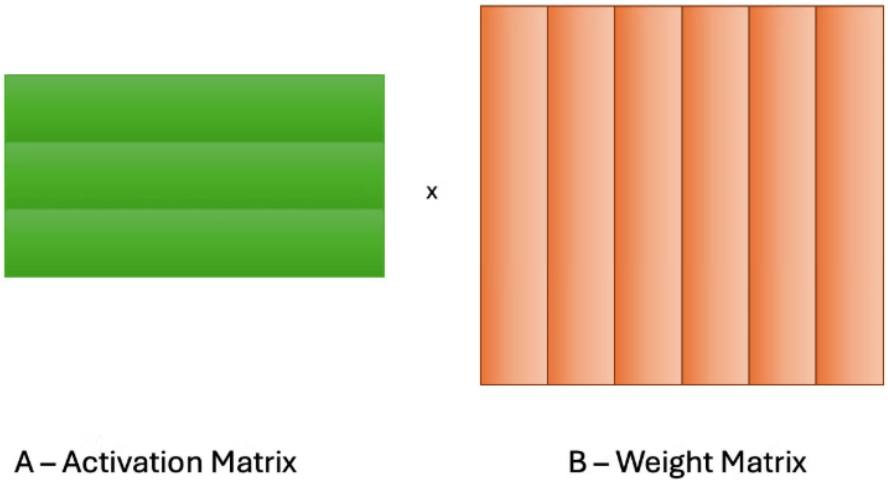
```

    return pid_m, pid_n
```

Listing 3: v2 列主序调度核心逻辑

4.3 优化原理

1. 提高 L2 缓存命中率，减少内存访问延迟



4.4 性能特点

- 在大规模 batch size 时性能最优

- 显著提高 L2 缓存命中率
- 适用于专家权重矩阵较大的场景

5 代码验证

5.1 性能测试

使用 PyTorch Profiler 测量三个版本的 CUDA 执行时间：

```
with torch.profiler.profile(
    activities=[torch.profiler.ProfilerActivity.CPU,
                torch.profiler.ProfilerActivity.CUDA],
    record_shapes=True,
    with_stack=True
) as prof:
    output = fused_moe_version(a, w1, w2, topk_weight, topk_ids,
        False)

print(prof.key_averages().table(sort_by="cuda_time_total",
    row_limit=10))
```

6 测试结果分析

6.1 性能数据汇总

根据 PyTorch Profiler 的输出，汇总了三个版本的性能数据：

表 1: 三个版本的 CUDA 执行时间对比（M=512, N=7168, K=4096）

版本	核心内核 CUDA 时间	总 CUDA 时间	相对性能
v0 (Grouped Launch)	4.083ms	4.392ms	基准
v1 (Split-K)	4.291ms	4.666ms	0.95x
v2 (Column Major)	0.992ms	1.304ms	3.37x

6.2 性能对比分析

1. **v2 vs v0**：性能提升 3.37x，接近 PyTorch 博客中报告的 4x 加速
2. **v1 vs v0**：性能略低 5%，可能是因为测试场景的矩阵配置不适合 Split-K
3. **v2 vs v1**：性能提升 3.58x，列主序调度优势明显

6.3 有关形状的调整

v0 中:

```
config = {
    'BLOCK_SIZE_M': 64,
    'BLOCK_SIZE_N': 64,
    'BLOCK_SIZE_K': 32,
    'GROUP_SIZE_M': 8
}

if topk_ids.numel() <= w1.shape[0]:#此时每个专家分配到的token极少
    config = {
        'BLOCK_SIZE_M': 16,
        'BLOCK_SIZE_N': 32,
        'BLOCK_SIZE_K': 64,
        'GROUP_SIZE_M': 1
    }
```

v2 中:

```
config = {
    'block_m': 64,
    'block_n': 64,
    'block_k': 32,
}

if topk_ids.numel() <= w1.shape[0]:
    config = {
```

```
            'block_m': 16,
            'block_n': 32,
            'block_k': 64,
        }
```

7 Cutlass 优化思路

Cutlass 作为 NVIDIA 的高性能 GEMM 库，在 MoE 优化方面支持上面提到的三种策略的组合（根据不同场景），同时还包含了其他优化策略。